

How the DFT and FFT Outputs Are Calculated

A step-by-step guide to the numerical multiplication and image-blurring pipelines

This document explains how the code in `bigmul.py`, `image_conv.py`, `transforms.py`, and `image_utils.py` produces the files under `outputs/`. It covers the mathematics, array operations, padding rules, saved images, reports, and verification.

Pipeline	Input	Main result
Task A	Two decimal strings	A decimal product in <code>product.txt</code>
Task B	A PNG image and blur kernel	<code>blurred.png</code> , <code>wraparound.png</code> , <code>kernel.png</code> , <code>comparison.png</code>

1. Shared Transform Core

The transform engine is selected by the caller. The naive DFT computes the definition directly. The radix-2 FFT computes the same mathematical transform using bit reversal and butterfly stages, reducing the work from $O(N^2)$ to $O(N \log N)$.

Forward DFT:

$$X[k] = \text{sum from } n=0 \text{ to } N-1 \text{ of } x[n] * \exp(-2\pi i * k * n / N)$$

Inverse DFT:

$$x[n] = (1/N) * \text{sum from } k=0 \text{ to } N-1 \text{ of } X[k] * \exp(+2\pi i * k * n / N)$$

- The FFT requires a power-of-two length, so the caller pads to `next_power_of_two` when necessary.
- The inverse FFT uses conjugated-sign twiddle factors and divides by N .
- For a 2D array, the transform is separable: transform every row, then every column.

2. Task A - Big-Integer Multiplication

The numerical product is calculated without multiplying the original huge integers directly. The decimal numbers are converted to polynomial coefficients, multiplied through the frequency domain, then converted back to decimal.

Step 1: Read the operands

Each input file contains two decimal strings. The sign is separated from the magnitude because the transform processes only nonnegative coefficients. The final sign is the product of the two operand signs.

Step 2: Convert to base-10⁴ limbs

Four decimal digits are packed into each coefficient. The coefficients are stored least-significant first.

```
123456789 -> [6789, 2345, 1]
```

The array represents the polynomial $6789 + 2345x + 1x^2$, evaluated at $x = 10^4$.

Step 3: Treat the limbs as polynomials

If A has limbs $a[0], a[1], \dots$ and B has limbs $b[0], b[1], \dots$, their product coefficients are the linear convolution:

$$c[k] = \text{sum over } i \text{ of } a[i] * b[k-i]$$

Step 4: Choose the transform length

For limb lengths L_a and L_b , the linear result needs $L_a + L_b - 1$ coefficients. FFT padding chooses the next power of two. This prevents the high-order coefficients from wrapping around as circular convolution.

```
For the first report: 3 + 3 - 1 = 5, so N = 8.
```

Step 5: Transform, multiply, and invert

- Zero-pad both limb arrays to N.
- Compute the transform of each array: $A[k]$ and $B[k]$.
- Multiply corresponding frequency values: $C[k] = A[k] * B[k]$.
- Apply the inverse transform to obtain the convolution coefficients.
- Take the real part and round, because floating-point arithmetic can leave tiny imaginary or fractional errors.

Step 6: Carry propagation

The convolution coefficients are not yet valid base-10⁴ digits. Each coefficient is combined with the incoming carry:

```
total = coefficient + carry; carry, digit = divmod(total, 10000)
```

The resulting digits are then joined from most significant to least significant, with four-digit zero padding between limbs. The original sign is restored, and the final string is saved in product.txt.

Step 7: Verification

Only after the custom calculation, the program compares the decimal result with Python's integer multiplication. MATCH means the two decimal strings are identical.

3. Task B - Image Blurring

The image pipeline applies a 2D convolution in the frequency domain. RGB images are processed as three independent planes; grayscale images use one plane.

Step 1: Load and normalize pixels

An 8-bit pixel is converted to a floating-point intensity in $[0, 1]$:

```
normalized_pixel = pixel_value / 255
```

Step 2: Construct and normalize the kernel

image_utils.make_kernel creates a box, Gaussian, bokeh, or motion kernel. The kernel is divided by its total sum so that all weights add to 1. This makes the operation an averaging or blur operation rather than a brightness amplifier.

Kernel	Meaning
Box	Every position in the square has equal weight.
Gaussian	Nearby pixels receive larger weights than distant pixels.
Bokeh	A filled circular aperture; bright points become discs.
Motion	A line-shaped streak representing camera movement.

Step 3: Determine the linear-convolution area

For image size $H \times W$ and kernel size $k_h \times k_w$, the full linear convolution has size:

```
(H + k_h - 1) x (W + k_w - 1)
```

For skyline_bokeh, $H = W = 512$ and $k_h = k_w = 19$, giving 530×530 .

Step 4: Zero-pad for FFT convolution

The image and kernel are placed in the upper-left corner of zero-filled arrays. With the radix-2 FFT, 530 is padded to 1024, so the actual transform size is 1024×1024 . The padding is what makes the result linear rather than circular.

Step 5: Transform rows and columns

The 2D transform is computed by applying the 1D engine to every row and then every column. This is valid because the 2D DFT is separable.

Step 6: Multiply spectra

For image spectrum $I(u,v)$ and kernel spectrum $K(u,v)$, the product spectrum is:

$$S(u, v) = I(u, v) * K(u, v)$$

The inverse 2D transform of S is the spatial convolution.

Step 7: Crop the centered result

The full result is larger than the original image. The program takes the $H \times W$ window starting at row $kh//2$ and column $kw//2$. This aligns the kernel center with each original pixel and preserves the input dimensions.

Step 8: Save the PNG

Values are clipped to $[0, 1]$, multiplied by 255, rounded, converted to uint8, and written as PNG. The internal calculation remains floating point; only the saved image is quantized to 8-bit values.

4. Linear Versus Circular Convolution

The program intentionally writes both versions so the padding effect can be observed.

blurred.png - linear convolution

- The image and kernel are zero-padded.
- Pixels outside the image are treated as zero.
- The full result is cropped back to the original image size.
- This is the normal, physically meaningful blur result.

wraparound.png - circular convolution

- The transform is performed at exactly $H \times W$, with no padding.
- The kernel is shifted so its center is at the transform origin.
- Content leaving the right edge re-enters at the left, and content leaving the bottom re-enters at the top.
- This creates visible boundary wraparound artifacts.

5. Verification of Image Results

The program takes the top-left 64×64 crop and calculates it in two independent ways: frequency-domain convolution and a direct four-loop spatial convolution.

```
direct[r,c] = sum over i,j of plane[r + kh//2 - i, c + kw//2 - j] * kernel[i,j]
```

Out-of-range source pixels are treated as zero. The maximum absolute difference between the two methods is reported. A value around 10^{-15} is normal floating-point roundoff; anything above 10^{-9} is classified as a bug.

The skyline report records:

Report field	Value
Image	512 x 512 RGB
Kernel	bokeh, 19 x 19
Linear-convolution size	530 x 530
Transform size	1024 x 1024
Maximum difference	3.331e-15
Verification	MATCH

6. Meaning of the Output Files

File	What it contains
product.txt	The final decimal product from Task A.
Task A report.txt	Digit counts, base, limb counts, transform length, product length, and verification.
blurred.png	Linear zero-padded blur, cropped to the original image size.
wraparound.png	Circular no-padding blur showing boundary wraparound.
kernel.png	Magnified visual preview of the normalized blur kernel.
comparison.png	Original, linear blur, and circular blur shown side by side.
Task B report.txt	Image dimensions, kernel, engine, padding dimensions, numerical error, and verification.

runtime plots	Timing curves for DFT, FFT, and direct spatial methods in the benchmark folders.
---------------	--

In short: Task A uses the convolution theorem to multiply large numbers as polynomials, while Task B uses the same theorem to blur image planes as 2D arrays. Zero-padding determines whether the convolution is linear or circular, and the direct calculations provide correctness checks for both pipelines.